

LECTURE

Why Secrets in Terraform Are Risky





Scenario



1

You're writing Terraform to create an RDS database, and it needs a password

2

You might think: "I'll just put it in a variable and move on"

3

But here's the reality - that password is about to show up in at least 3 places you might not expect



The Core Problem

Terraform needs secrets to manage infrastructure, but those secrets can be exposed in multiple places



Secrets Stored as Plaintext

API Keys, Passwords, Account Numbers, etc. could end up in files (not good).



Secrets Shared Across Teams

Remote backends make state accessible to multiple people and logs and outputs are captured.



Secrets Need Protection at Every Layer

Multiple exposure points means multiple protections are needed. Understanding WHERE secrets appear is the first step.

```
terraform.tfstate

1 {
2   "version": 4,
3   "terraform_version": "...",
4   "serial": ...,
5   "lineage": "...",
6   "outputs": {
7     "database_password_output": {
8       "value": "MySuperSecretPassword123!",
9       "type": "string",
10      "sensitive": true
11    }
12  },
13  "resources": [
14    {
15      "mode": "managed",
16      "type": "aws_db_instance",
17      "name": "main",
18      "provider": "...",
19      "instances": [
20        {
21          "schema_version": ...,
22          "attributes": {
23            // ... other attributes
24            "password": "MySuperSecretPassword123!",
25            // ...
26          },
```



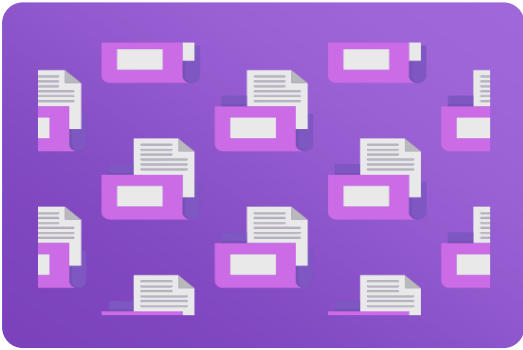
“ ”

This is not a bug – Terraform NEEDS this information to manage your infrastructure.

We just ways to protect it and prevent it.

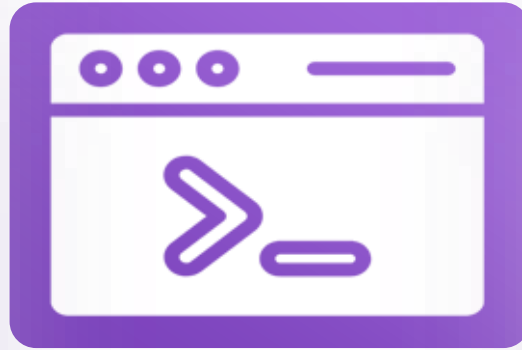


Where Sensitive Data Can End Up



State Files

- Y terraform.tfstate (local or remote)
- Y Plaintext JSON with resource attributes



Logs & Console

- Y terraform plan output
- Y Provider error messages
- Y CI/CD build logs



Version Control

- Y Committed .tfvars files
- Y Hardcoded values in .tf files
- Y Git history (permanent)



Plan Files

- Y Saved terraform plan files
- Y Contain proposed changes with values
- Y Often stored in CI/CD artifacts

LECTURE

Protecting Secrets in Your Configuration



Protecting Secrets in Your Configuration

Overview



Sensitive Variables

Hide Values in
Output and Logs



Environment Variables

Keep Secrets out of
Version Control



External Secret Sources

Don't Store Secrets
in Terraform at all

These Techniques Work Together for Security in Layers

The Problem:

Passwords in Plain Sight

```
1 variable "db_password" {
2   description = "Database password"
3   type        = string
4 }
5
6 resource "aws_db_instance" "main" {
7   identifier      = "mydb"
8   engine          = "postgres"
9   instance_class  = "db.t3.micro"
10  username        = "admin"
11  password        = var.db_password
12 }
```



TERMINAL

```
$ terraform plan

Terraform will perform the following
actions:

# aws_db_instance.main will be created
+ resource "aws_db_instance" "main" {
+   password = "MyS3cr3tP@ssw0rd!"
+   username = "admin"
+   ...
+ }
```

Plan: 1 to add, 0 to change, 0 to destroy.

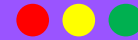
This information up in CI/CD logs, console output, and anywhere terraform plan runs

The Solution:

Mark Variables as Sensitive



```
1
2
3
4
5
6
7 resource "aws_db_instance" "main" {
8     identifier      = "mydb"
9     engine           = "postgres"
10    instance_class  = "db.t3.micro"
11    username         = "admin"
12    password         = var.db_password
13 }
```



TERMINAL

```
$ terraform plan

Terraform will perform the following
actions:

# aws_db_instance.main will be created
+ resource "aws_db_instance" "main" {
+   password      = (sensitive value)
+   username      = "admin"
+   ...
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

What Sensitive Variables Protect



✓ What It DOES Protect

- ✓ Hides values in `terraform plan` output
- ✓ Hides values in `terraform apply` output
- ✓ Prevents values from appearing in logs
- ✓ Redacts values in error messages
- ✓ Works for variables and outputs

✗ What It DOESN'T Protect

- ✗ Still stored in state file (plaintext)
- ✗ Still visible if you open state directly
- ✗ Still transmitted to providers
- ✗ Not encryption - just output masking

Think of sensitive variables as 'hide from logs' and not 'encrypt the data'

Sensitive Outputs



```
1 output "database_endpoint" {
2   description = "Database connection endpoint"
3   value       = aws_db_instance.main.endpoint
4 }
5
6 output "database_password" {
7   description = "Database password"
8   value       = aws_db_instance.main.password
9   sensitive   = true # ← Prevents output from showing in console
10 }
```



TERMINAL

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

```
database_endpoint = "mydb.abc123.us-east-1.rds.amazonaws.com:5432"
database_password = <sensitive>
```

Sensitive Outputs



Regular outputs show their values



Sensitive outputs are hidden



Still accessible programmatically or using
terraform output <name> **command**

```
1  output "database_endpoint" {
2      description = "Database connection endpoint"
3      value       = aws_db_instance.main.endpoint
4  }
5
6  output "database_password" {
7      description = "Database password"
8      value       = aws_db_instance.main.password
9      sensitive   = true
10 }
```

The Problem:

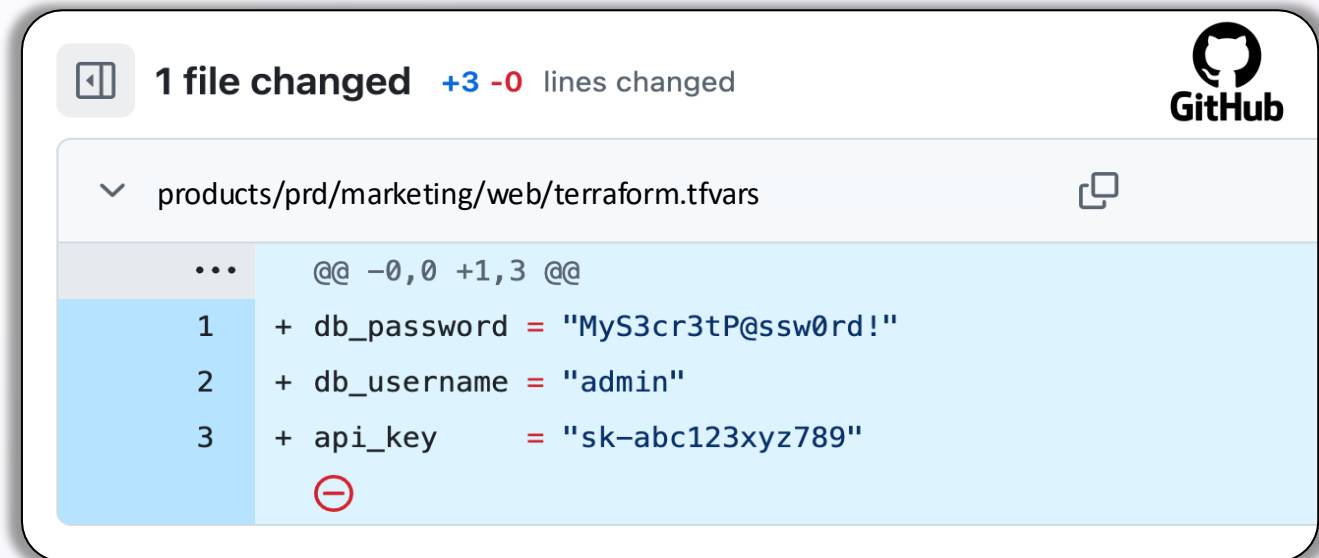
Committing Secrets to Version Control



✗ Don't Do This

terraform.tfvars

```
1 db_password = "MyS3cr3tP@ssw0rd!"
2 db_username = "admin"
3 api_key     = "sk-abc123xyz789"
```



because it ends up here...forever

The Solution: Environment Variables



TERMINAL

```
$ export TF_VAR_db_password="MyP@ssw0rd!"
$ export TF_VAR_api_key="sk-abc123xyz"

$ terraform apply
...
```



```
1 variable "db_password" {
2   description = "Database password"
3   type        = string
4   sensitive   = true
5 }
```



Set the Value using an Environment Variable Instead

✦ No default value needed!

✦ Terraform automatically reads `TF_VAR_db_password`

Benefits of Using Environment Variables



✓ Never stored in files

✓ Perfect for CI/CD pipelines

✓ Never committed to git

✓ Works with sensitive = true

✗ Does NOT keep secrets out of state

Environment Variables in GitHub Actions



```
1  name: Terraform Apply
2  on: [push]
3
4  jobs:
5    terraform:
6      runs-on: ubuntu-latest
7      steps:
8        - uses: actions/checkout@v2
9
10       - name: Terraform Apply
11         env:
12           TF_VAR_db_password: ${ secrets.DB_PASSWORD }
13           TF_VAR_api_key: ${ secrets.API_KEY }
14         run: |
15           terraform init
16           terraform apply -auto-approve
```

Secrets stored in GitHub Secrets

Injected as env vars at runtime

Never appear in logs or code

The Best Practice: Don't Store Secrets in Terraform



**Secrets Managed by
Dedicated Tools**

**Centralized Rotation and
Access Control**

**Reduced Secrets in
Terraform State**

**One Source of Truth
for Secrets**



Reading Secrets from AWS Secrets Manager

```
1  # Read the secret from AWS Secrets Manager
2  data "aws_secretsmanager_secret_version" "db_pass" {
3    secret_id = "production/database/password"
4  }
5
6  # Use it in your resource
7  resource "aws_db_instance" "main" {
8    identifier      = "mydb"
9    engine          = "postgres"
10   instance_class  = "db.t3.micro"
11   username        = "admin"
12   # Secret is fetched at apply time
13   password = data.aws_secretsmanager_secret_version.db_pass.secret_string
14 }
```

- ✦ Terraform reads the secret when it runs
- ✦ Password is NOT in your .tf files
- ✦ Still ends up in state
- ✦ Uses a provider for the target secrets manager

Similar patterns work with Azure Key Vault, GCP Secret Manager, and HashiCorp Vault

Protection Quick Reference



Technique	Protects Logs/Output?	Protects State?	Protects Git Commits?	Best For
Sensitive Variables	✅ Yes	❌ No	❌ No	Hiding output, marking intent
Environment Variables	🌟 Partial	❌ No	✅ Yes	Local dev, CI/CD, keeping secrets out of files
External Sources	🌟 Partial	!! Reduces	✅ Yes	Production secrets, centralized management

- ❧ Create environment vars to get secrets into Terraform
- ❧ Store secrets in external sources where possible
- ❧ Use sensitive variables to hide output

LECTURE

Securing State Files



The State File Problem



Terraform state contains complete resource configuration



Includes all sensitive values (passwords, keys, tokens)



Stored as plaintext JSON by default



No file encryption unless you configure it yourself

```
terraform.tfstate

1  {
2    "version": 4,
3    "terraform_version": "...",
4    "serial": ...,
5    "lineage": "...",
6    "outputs": {
7      "database_password_output": {
8        "value": "MySuperSecretPassword123!",
9        "type": "string",
10       "sensitive": true
11      }
12    },
13    "resources": [
14      {
15        "mode": "managed",
16        "type": "aws_db_instance",
17        "name": "main",
18        "provider": "...",
19        "instances": [
20          {
21            "schema_version": ...,
22            "attributes": {
23              // ... other attributes
24              "password": "MySuperSecretPassword123!",
25              // ...
26            }
27          }
28        ]
29      }
30    ]
31  }
```

Why Local State is Risky



Local State

(terraform.tfstate in your directory)

VS

Remote State

(S3, HCP Terraform, etc.)

- ✗ Stored on your laptop unencrypted
- ✗ Easy to accidentally commit to git
- ✗ Hard to share with teams safely
- ✗ No access controls
- ✗ No encryption at rest

- ✓ Stored in a centralized location for team access
- ✓ Never stored on a local filesystem (laptop)
- ✓ Can enable encryption at rest (most backends)
- ✓ Access is controlled via IAM/RBAC systems
- ✓ Audit Logging is generally available

Always use remote state with encryption for production workloads

Configuring Encrypted Remote State



```
1 terraform {  
2   backend "s3" {  
3     bucket      = "terraform-state"  
4     key         = "prod/network/terraform.tfstate"  
5     region      = "us-east-1"  
6     use_lockfile = true  
7     encrypt     = true      # server-side encryption  
8     kms_key_id  = "<kms_id>" # kms key encryption  
9   }  
10 }
```

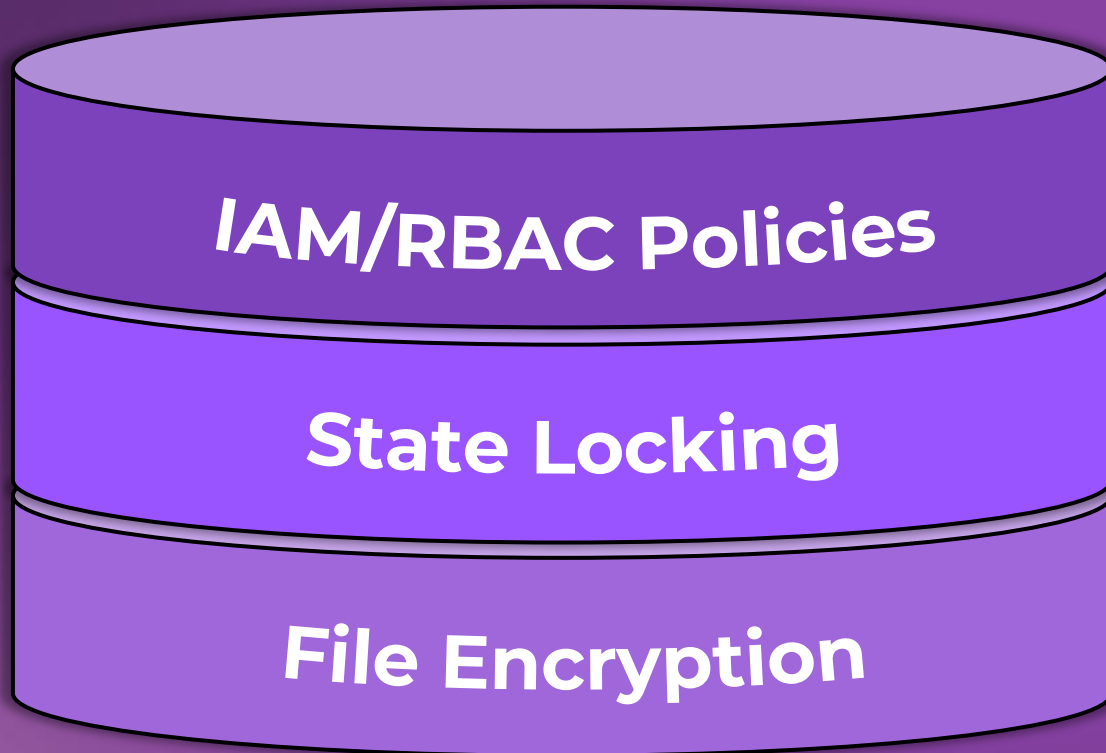
`encrypt = true` **enables AES-256 encryption**

State is encrypted before being written to S3

State locking enabled to prevent corruption

Access controlled via IAM policies

Controlling Who Can Access Terraform State



- ⚡ Only authorized users/roles can read state
- ⚡ Separate read vs write permissions
- ⚡ Audit logging for all access
- ⚡ Prevents concurrent modifications
- ⚡ Lockfile for S3, built-in for others
- ⚡ Protects against corruption
- ⚡ KMS for additional encryption layer
- ⚡ Customer-managed keys for compliance
- ⚡ Automatic key rotation

Defense in Depth: Encryption + Access controls + Audit logging

State File Security Checklist



- ☒ Always use remote state for production
- ☒ Enable encryption at rest
- ☒ Implement access controls
- ☒ Enable state locking



State files will contain secrets. Your job is to protect where they're stored and who can access them.

LECTURE

Keeping Secrets Out of State

Ephemeral Values and Write-only
Arguments



Even with All Our Protections...

✓ Sensitive variables → Hide output

✓ Environment variables → Avoid git

✓ Encrypted state → Protect storage

✓ External sources → Best practice



Sensitive Values still end up in state.
But...what if we could change that?

```
terraform.tfstate

1 {
2   "version": 4,
3   "terraform_version": "...",
4   "serial": ...,
5   "lineage": "...",
6   "outputs": {
7     "database_password_output": {
8       "value": "MySuperSecretPassword123!",
9       "type": "string",
10      "sensitive": true
11    }
12  },
13  "resources": [
14    {
15      "mode": "managed",
16      "type": "aws_db_instance",
17      "name": "main",
18      "provider": "...",
19      "instances": [
20        {
21          "schema_version": ...,
22          "attributes": {
23            // ... other attributes
24            "password": "MySuperSecretPassword123!",
25            // ...
26          },
```



Ephemeral Values

Secrets That Don't Persist

Introduced in Terraform 1.10, ephemeral values are a new approach to handling secrets. They exist only in memory during Terraform operations and are never written to state files or plan files. Values are calculated fresh on each Terraform run and discarded immediately after use.

Ephemeral variables must be marked with `ephemeral = true`

TERRAFORM 1.10+ REQUIRED

```
1 variable "database_password" {
2   description = "Password for the database"
3   type        = string
4   sensitive    = true
5   ephemeral    = true
6 }
7
```

Omit Value from State and Plans

Set the `ephemeral` argument to `true` in a `variable` block containing temporary sensitive data to omit that value from state and plan files.

Note: The value is also redacted in the Terraform CLI and the HCP Terraform UI



Where Can I Use Ephemeral Values?

Local
Values

Provider
Blocks

Ephemeral
Resources

Ephemeral
Variables

Ephemeral
Outputs

Provisioner
Blocks

Write-Only
Arguments



Important Restriction

Where You CAN'T Use Ephemeral Values

```

# This DOESN'T work ❌
resource "azurerm_mssql_server" "db" {
  administrator_login_password = var.db_password # ephemeral
  # ...
}

Error: Invalid use of ephemeral value
| ...
|
| Ephemeral values are not valid in resource arguments,
| because resource instances must persist between
| Terraform phases.
```

- ❖ Resources need values that persist in state.
- ❖ But...we know that ephemeral values disappear after use.
- ❖ So how do we use secrets in resources?

This is where write-only arguments come in...



```
1  variable "github_owner" {
2      type          = string
3      description = "GitHub org"
4  }
5
6  variable "github_token" {
7      type          = string
8      description = "GitHub token"
9      sensitive     = true
10     ephemeral     = true
11 }
12
13 provider "github" {
14     owner = var.github_owner
15     token = var.github_token
16 }
```

Use Case for Ephemeral Values

The sensitive helps prevent the token from being displayed in output.

Ephemeral is the key part: Terraform omits the value from state and plan files entirely.

Ephemeral Resources

- ✧ New block type (not data or resource)
- ✧ Fetches data at runtime and stored in memory
- ✧ Never persisted in state and flushed when Terraform operations are completed
- ✧ Opened and closed during execution



```
1  # Regular data source (stored in state)
2  data "azurerm_key_vault_secret" "db" {
3      name          = "database-password"
4      key_vault_id = var.key_vault_id
5  }
6
7  # Ephemeral resource (NOT stored in state)
8  ephemeral "azurerm_key_vault_secret" "db" {
9      name          = "database-password"
10     key_vault_id = var.key_vault_id
11 }
```

- ✧ Uses `ephemeral` keyword instead of `data`
- ✧ Syntax is nearly identical to data sources

When Should I Use an Ephemeral Resource?



Fetching secrets from Vault

Reading temporary credentials

Retrieving dynamic tokens



The provider developer determines which arguments you can define for an ephemeral resource.



Write-Only Arguments

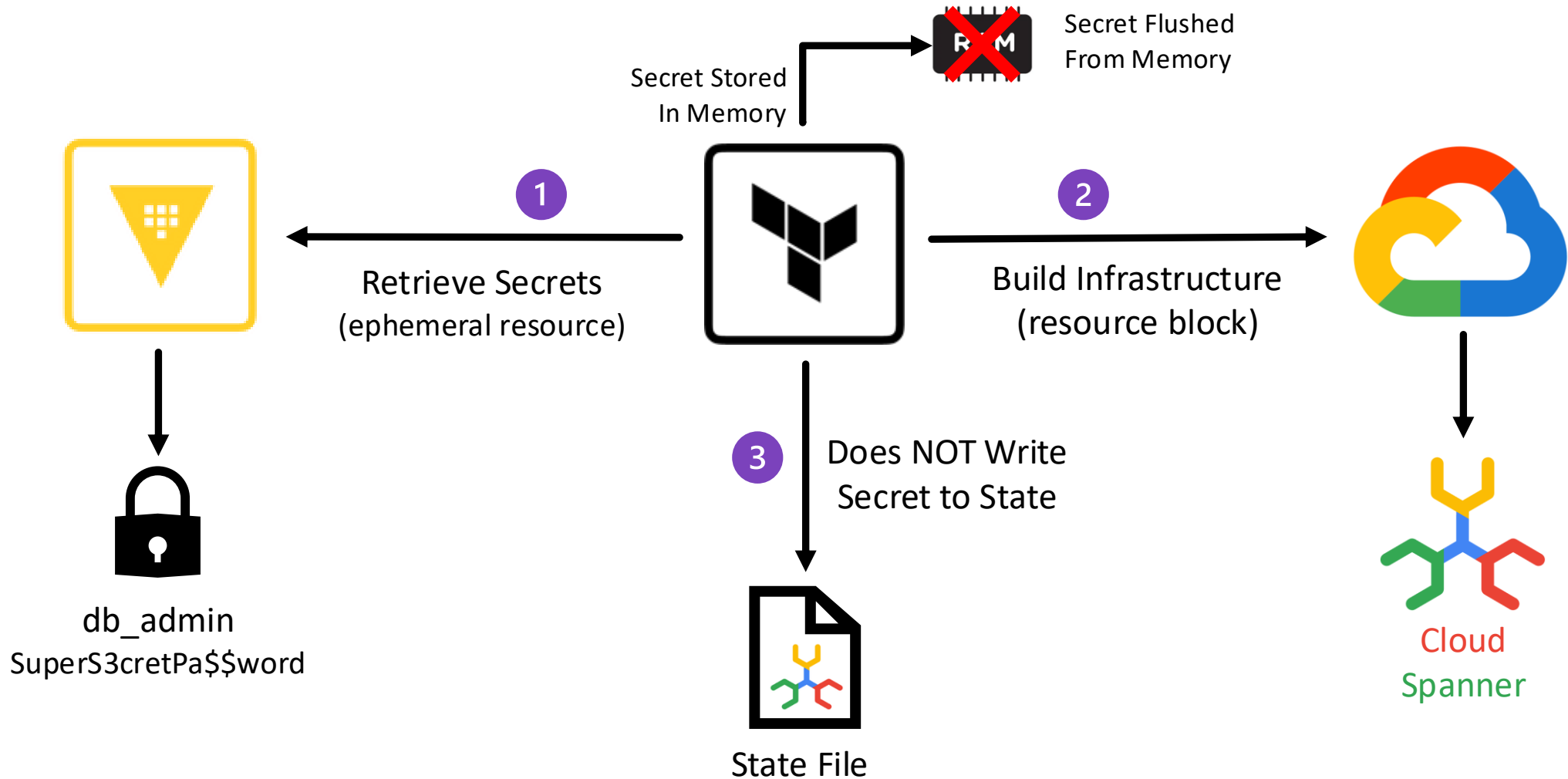
Write-only arguments let you securely pass temporary values to Terraform's managed resources during an operation without persisting those values to state or plan files.

Each provider defines any available write-only arguments on their managed resources.



```
1 resource "aws_db_instance" "test" {  
2     instance_class      = "db.t3.micro"  
3     allocated_storage   = "5"  
4     engine              = "postgres"  
5     username            = "example"  
6     skip_final_snapshot = true  
7     password_wo         = "old-password-here"  
8     password_wo_version = 1  
9 }
```

Write-Only Arguments



Comparison



Regular Resource Block

```
1 resource "aws_db_instance" "test" {
2   instance_class      = "db.t3.micro"
3   allocated_storage   = "5"
4   engine               = "postgres"
5   skip_final_snapshot = true
6   username             = "db-admin"
7   password             = var.database_password
8 }
9
```

Secret Would Be Stored in State

Write-only Argument with Versions

```
1 resource "aws_db_instance" "test" {
2   instance_class      = "db.t3.micro"
3   allocated_storage   = "5"
4   engine               = "postgres"
5   skip_final_snapshot = true
6   username             = "db-admin"
7   password_wo          = var.ephemeral_pass
8   password_wo_version = 1
9 }
```

Secret Would NOT Be Stored in State

LECTURE

Working with HashiCorp Vault for Dynamic Credentials

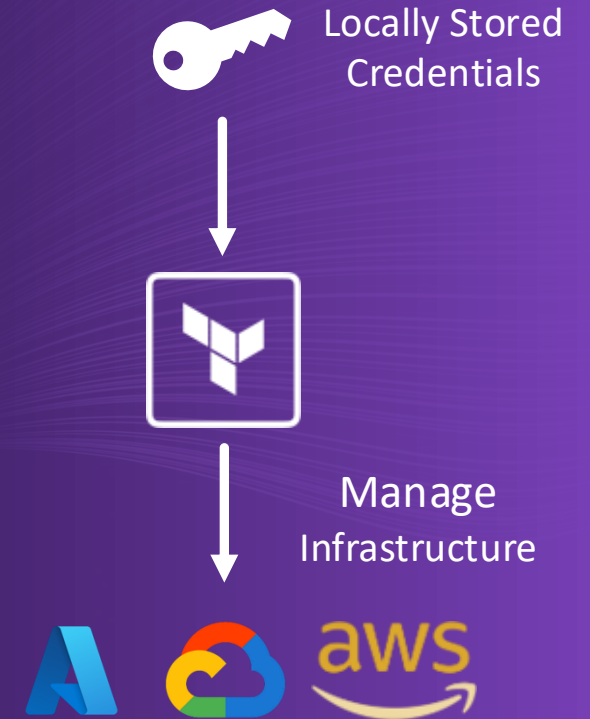


The Problem with Using Static Credentials



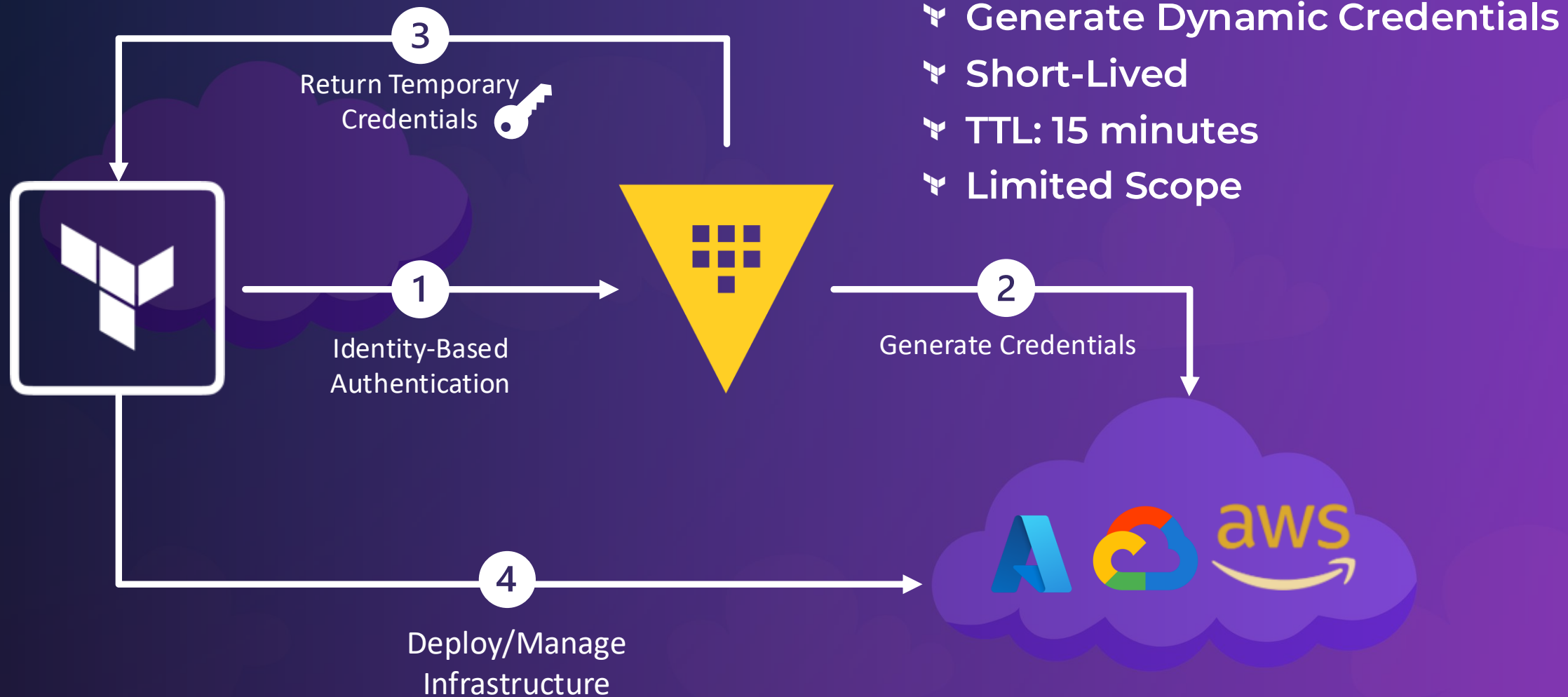
Traditional Approach:

- ❖ Terraform users get their own, long-lived credentials
- ❖ Stored locally on machines/laptops (security risk)
- ❖ Difficult to rotate, revoke, and audit
- ❖ Large attack surface because they are highly privileged



What if credentials were short-lived and generated on-demand?

Dynamic Credential Generation





```
1 provider "vault" {
2   address = "https://vault.example.com:8200"
3
4   <identity-based authentication config>
5   . . .
6 }
7
8 ephemeral "vault_aws_access_credentials" "creds" {
9   backend = vault_aws_secret_backend.aws.path
10  role    = vault_aws_secret_backend_role.example.name
11  type    = "creds"
12  region  = "us-east-1"
13 }
14
15 provider "aws" {
16   region = "us-east-1"
17   access_key = ephemeral.vault_aws_access_credentials.creds.access_key
18   secret_key = ephemeral.vault_aws_access_credentials.creds.secret_key
19 }
20
21 resource "aws_instance" "main" {
22   # ... provisions with temporary credentials
23 }
```

The Vault Provider Workflow

- ❧ Credentials generated when requested
- ❧ Valid for 120 seconds
- ❧ Automatically revoked after TTL
- ❧ Creds not written to state when using ephemeral resources

Summary: Why Use Vault for Terraform



Centralized Secret Management

- ❏ One place to manage platform credentials
- ❏ Easy to rotate long-lived credentials

Reduced Attack Surface

- ❏ Credentials expire automatically
- ❏ Compromised creds only valid for minutes

No Local Credential Storage

- ❏ Developers never touch AWS credential
- ❏ No credential redistribution